jb, had, jvo

Lab 2: Invariantes

Este lab deve ser feito individualmente.

Submeta o seu código ao mooshak http://deei-mooshak.ualg.pt/~jvo/ usando o seu login da ualg (sem @ualg.pt). Ex: a123456

Uma submissão permanecerá *pending* até que seja solicitada a sua validação ao professor durante a aula prática. Só as submissões *final* serão consideradas para avaliação.

A submissão deverá ser feita até

23 de fevereiro 2026

A validação poderá ser feita posteriormente, se necessário, até **12** de março de 2026

NB: Nos problemas seguintes, sempre que necessário, considera-se que dois double *d* e *g* são iguais se $|d-g|<1e-9$

Consulte agora os slides da Teóricas 2 e 3 antes de responder aos problemas seguintes.

Problema D: Segmento de reta

Um segmento de reta pode ser definido pelo ponto de um dos seus extremos, uma direção dada por um vetor posição e um comprimento.

A tarefa deste problema é desenvolver a classe SegmentoReta, que constrói uma nova instância a partir de uma instância da classe Ponto e uma instância de um Vetor posição, tal como indicado acima. O comprimento do segmento é dado pelo módulo do Vetor posição.

O Segmento não pode ter os dois extremos no mesmo ponto. Se tal suceder, a condição invariante de instância é violada, sendo impresso "SegmentoReta:iv".

Este programa deve ter como base as classes implementadas no Lab 1. Se não tiver, independentemente do mérito da implementação, terá avaliação zero.

Restrições complementares

Nota: Submissão não aceites tem avaliação zero. O não cumprimento de cada uma das restrições abaixo para uma submissão Accepted implica uma redução de 20% na nota da submissão. No limite a nota será zero se todas as restrições não forem cumpridas mesmo que a submissão tenha sido aceite pelo Mooshak.

Seguindo os princípios da programação orientada por objetos, o programa deverá estar estruturado segundo um conjunto de classes apropriadas, nomeadamente:

1) O programa deve usar e atualizar a classe Vetor do lab 1, para imprimir a mensagem de erro "Vetor:iv", se a invariante de instância for violada. Deve também adicionar-se um construtor a Vetor para criar um Vetor a partir de um Ponto.

2) Deve, assim, ser criada a classe Ponto, com um construtor que cria um ponto a partir de duas coordenadas cartesianas *x* e *y*, ambas do tipo double. A impressão das coordenadas de Ponto é efetuada pelo método **toString()** da classe Ponto, no formato: (*x*,*y*).

3) Deve também ser criada a classe SegmentoReta, com um construtor que recebe um Ponto e um Vetor. A impressão dos extremos do segmento de reta, pontos A e B, é efetuada pelo método **toString()** desta classe, no formato: sr((x_A,y_A); (x_B,y_B))

4) Todas as variáveis de instância das classes tem de ter visibilidade private, sendo acedidas apenas por assessores de leitura. Não podem ter assessores de escrita.
A classe **Cliente** apenas lê os dados de entrada, processa-os chamando apenas os métodos necessários das classes **Vetor**, **Ponto** e **SegmentoReta** e escreve o resultado na consola.

5) Nos comentários indique:
   i) Uma linha com a descrição da responsabilidade da classe ou do que o método faz
   ii) @author (apenas para classes)
   iii) @version (apenas para classes; inclua uma data)
   iv) @inv (apenas para classes; inclua uma descrição da invariante usada)
   v) @param (apenas para métodos e construtores)
   vi) @return (apenas para métodos)
   vii) @see (qualquer referência bibliográfica ou sítio web consultado para o desenvolvimento do código respetivo)

Entrada
A entrada é composta por duas linhas.
A primeira linha contém dois doubles, x_o e y_o, que representam as coordenadas cartesianas do ponto que representa um dos extremos do segmento de reta.
A segunda linha contém dois doubles, x_v e y_v, que representam as coordenadas cartesianas do ponto que define o vetor posição.

Saída
Uma linha com uma das opções seguintes:

i) "Vetor:iv"; mensagem indicando que o Vetor tem módulo com valor zero.
ii) "SegmentoReta:iv"; indicação de que o segmento de reta tem comprimento com valor zero.
iii) As coordenadas do ponto que define o extremo do SegmentoReta mais próximo da origem, seguidas por um espaço e pelas coordenadas do ponto que define o outro extremo. As coordenadas são doubles com duas casas decimais de precisão. Exemplo: sr((1,1); (3,3))

Exemplo de Entrada 1
2 2
-1 0

Exemplo de Saída 1
sr((1.00,2.00); (2.00,2.00))

Exemplo de Entrada 2
3 3
2 4.5

Exemplo de Saída 2
sr((3.00,3.00); (5.00,7.50))

Exemplo de Entrada 3
2 3
0 0

Exemplo de Saída 3
Vetor:iv

Problema E: Interseção entre um Vetor e um Segmento de reta

Desenvolva um programa que dado um Vetor, conforme definido nos problemas anteriores, e um segmento de reta definido por dois pontos com coordenadas cartesianas, determine se o segmento interseta ou não o Vetor ou indique se existiu alguma violação da condição invariante de instância em alguma das classes.

Este programa deve usar e desenvolver as classes definidas no problema D. Se não o fizer, independentemente do mérito da implementação, terá avaliação zero.

Restrições complementares
Nota: Submissão não aceites tem avaliação zero. O não cumprimento de cada uma das restrições abaixo para uma submissão Accepted implica uma redução de 20% na nota da submissão. No limite a nota será zero se todas as restrições não forem cumpridas mesmo que a submissão tenha sido aceite pelo Mooshak.

Seguindo os princípios da programação orientada por objetos, o programa deverá estar estruturado segundo um conjunto de classes apropriadas, nomeadamente:

1) O programa deve usar como base as classes definidas no problema anterior deste lab, o problema D e adicionar a SegmentoReta o método **Point intersect(Vetor v)** que retorna o ponto de interseção ou **null**.

2) Deve ser adicionado à classe Vetor o método **Point intersect(SegmentoReta v)** que retorna o ponto de interseção ou **null**, **sem duplicar o código implementado no método intersect() de SegmentoReta, referido acima**.

3) Deve adicionar-se um construtor a SegmentoReta que cria uma nova instância a partir de duas instâncias de Ponto.

4) Todas as variáveis de instância das classes têm de ter visibilidade private, sendo acedidas apenas por assessores de leitura. Não podem ter assessores de escrita.
A classe **Cliente** apenas lê os dados de entrada, processa-os chamando apenas os métodos necessários das classes **Vetor**, **Ponto** e **SegmentoReta** e escreve o resultado na consola.

5) Nos comentários indique
   i) Uma linha com a descrição da responsabilidade da classe ou do que o método faz
   ii) @author (apenas para classes)
   iii) @version (apenas para classes; inclua uma data)
   iv) @inv (apenas para classes; inclua uma descrição da invariante usada)
   v) @param (apenas para métodos e construtores)
   vi) @return (apenas para métodos)
   vii) @see (qualquer referência bibliográfica ou sitio web consultado para o desenvolvimento do código respetivo)

Entrada
A entrada é composta por duas linhas.
A primeira linha contém quatro doubles, separados por espaços, que representam, respetivamente, as coordenadas (x,y) do primeiro e do segundo ponto que definem os extremos do segmento: x_A y_A x_B y_B. A segunda linha contém dois doubles, x_v e y_v, que representam as coordenadas cartesianas do ponto que define o vetor posição.

Saída
Uma linha com uma das opções seguintes:

i) "Vetor:iv"; mensagem indicando que o Vetor tem módulo com valor zero.
ii) "SegmentoReta:iv"; indicação de que o segmento de reta tem comprimento com valor zero.
iii) As coordenadas do ponto de interseção no formato (x,y) com 2 casas decimais de precisão, ou "null" se o Vetor e o Segmento de reta não se intersectarem.

Exemplo de Entrada 1
0 1 4 1
2 2

Exemplo de Saída 1
(1.00,1.00)

Exemplo de Entrada 2
4 4 6 6
2 2

Exemplo de Saída 2
null

Exemplo de Entrada 3
2 2 2 2
1 4

Exemplo de Saída 3
SegmentoReta:iv

Exemplo de Entrada 4
2 0 -1.2 0.4
0 0

Exemplo de Saída 4
Vetor:iv